SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 03	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills		
Student Name:	V. Tejaswini	**Reg. No.:**	192424360
Date:	3-08-2026	**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Architecture Design Assignment

Question

Based on your assigned problem statement, perform an Architecture Design Analysis and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.

The Smart Waste Collection Monitoring and Route Optimization Platform is designed to modernize waste management by using IoT sensors, GPS tracking, cloud computing, and data analytics. The main objective is to monitor garbage bin levels in real time, automatically notify collection authorities when bins become full, and generate optimized routes for waste collection vehicles. This system helps reduce unnecessary trips, fuel consumption, operational costs, and environmental pollution while ensuring cleaner cities.

The platform also provides real-time monitoring dashboards for municipal authorities, enabling them to make informed decisions based on live data. Citizens can report overflowing bins or cleanliness issues through a mobile application, improving communication between the public and waste management departments. Overall, the system improves efficiency, transparency, and sustainability in urban waste management.

System Objectives

- Monitor garbage bin fill levels in real time.
- Detect overflowing bins automatically.
- Optimize waste collection routes using GPS.
- Reduce fuel consumption and operational costs.
- Improve city cleanliness and public hygiene.
- Enable citizens to report waste-related issues.
- Provide live dashboards for municipal authorities.
- Generate reports and analytics for better planning.
- Improve resource utilization.

- Support smart city initiatives.

  - Analyze the functional and non-functional requirements that influence the software architecture.
  - The system should allow users, drivers, and administrators to securely register, log in, and access features based on their assigned roles.
  - Smart waste bins equipped with IoT sensors should continuously monitor the fill level and automatically send data to the central server.
  - The platform should display the real-time status and location of all waste bins on a dashboard, helping authorities identify bins that require immediate attention.
  - The system should generate automatic alerts and notifications whenever a bin reaches its maximum capacity or requires maintenance.
  - It should use GPS tracking and route optimization algorithms to suggest the shortest and most efficient collection routes for waste collection vehicles.
  - Citizens should be able to report overflowing bins or sanitation issues through a mobile or web application, along with photos and location details.
  - The system should maintain a central database to store user information, sensor data, collection history, complaints, and reports.

    Administrators should be able to monitor vehicle movements, collection progress, and system performance through interactive dashboards and analytics.
  - The platform should generate daily, weekly, and monthly reports on waste collection activities, vehicle usage, and operational efficiency.
  - The system should support integration with external services such as GPS, mapping APIs, SMS gateways, and email notification service
  - Non-Functional Requirements (Medium Points)
  - The system should provide fast response times so that users can access real-time waste monitoring information without delays.
  - It should ensure high availability, allowing the platform to operate continuously with minimal downtime, even during peak usage.
  - Strong security mechanisms, including user authentication, data encryption, and role-based access control, should protect sensitive information.

- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

Key Quality Attributes

Scalability

- Supports thousands of smart bins and users.
- Easily adds new vehicles and collection zones.
- Allows expansion to multiple cities.
- Cloud infrastructure handles increasing workloads.
- Maintains performance during peak usage.

Maintainability

- Modular architecture simplifies updates.
- Individual services can be modified independently.
- Easy debugging and testing of components.
- Supports future feature additions.
- Reduces software maintenance time and cost.

Reliability

- Provides accurate real-time sensor data.
- Ensures continuous system operation.
- Detects and handles failures automatically.
- Prevents data loss through regular backups.
- Delivers consistent and dependable performance.

Availability

- Operates 24×7 with minimal downtime.
- Uses cloud servers for high availability.

- Supports automatic failover during failures.
- Enables uninterrupted monitoring of waste bins.
- Ensures continuous access for all users.

Performance

- Processes sensor data in real time.
- Generates optimized routes quickly.
- Responds to user requests with low latency.
- Efficiently handles multiple simultaneous users.
- Optimizes database and network usage.

Security

- Uses secure user authentication and login.
- Encrypts sensitive data during transmission and storage.
- Implements role-based access control.
- Protects against unauthorized access and cyber threats.
- Maintains user privacy and data confidentiality.

2. Select a Suitable Software Architecture

  - Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).

Selected Architecture: Hybrid Architecture (Layered + Microservices + Event-Driven)

The Smart Waste Collection Monitoring and Route Optimization Platform uses a Hybrid Architecture because it combines the strengths of multiple architectural styles. The Layered Architecture organizes the system into presentation, business logic, and data layers, making development and maintenance easier. Microservices Architecture divides the application into independent services such as user management, bin monitoring, route optimization,

notifications, and reporting. Event-Driven Architecture enables the system to process real-time events generated by IoT sensors whenever waste bins become full or require maintenance.

This architecture is highly suitable for smart city applications because it supports real-time monitoring, cloud deployment, easy scalability, and efficient communication between multiple components.

Features of the Selected Architecture

- Combines Layered, Microservices, and Event-Driven architectures.
- Supports real-time IoT sensor data processing.
- Allows independent deployment of services.
- Provides better scalability and flexibility.
- Simplifies maintenance and future upgrades.
- Enables seamless integration with GPS and cloud services.
- Improves system reliability and performance.

  - Justify why the selected architecture is the most suitable for the given problem.

The Hybrid Architecture is the most suitable choice because the platform needs to manage thousands of smart bins, GPS-enabled vehicles, mobile applications, cloud databases, and real-time notifications simultaneously. A single architectural style may not efficiently handle all these requirements, whereas a hybrid approach provides the advantages of multiple architectures.

The layered structure improves organization by separating the user interface, business logic, and database operations. Microservices allow each module, such as route optimization, complaint management, and notifications, to operate independently, making the system easier to maintain and scale. The event-driven model ensures that IoT sensor data is processed immediately, allowing quick notifications and optimized waste collection.

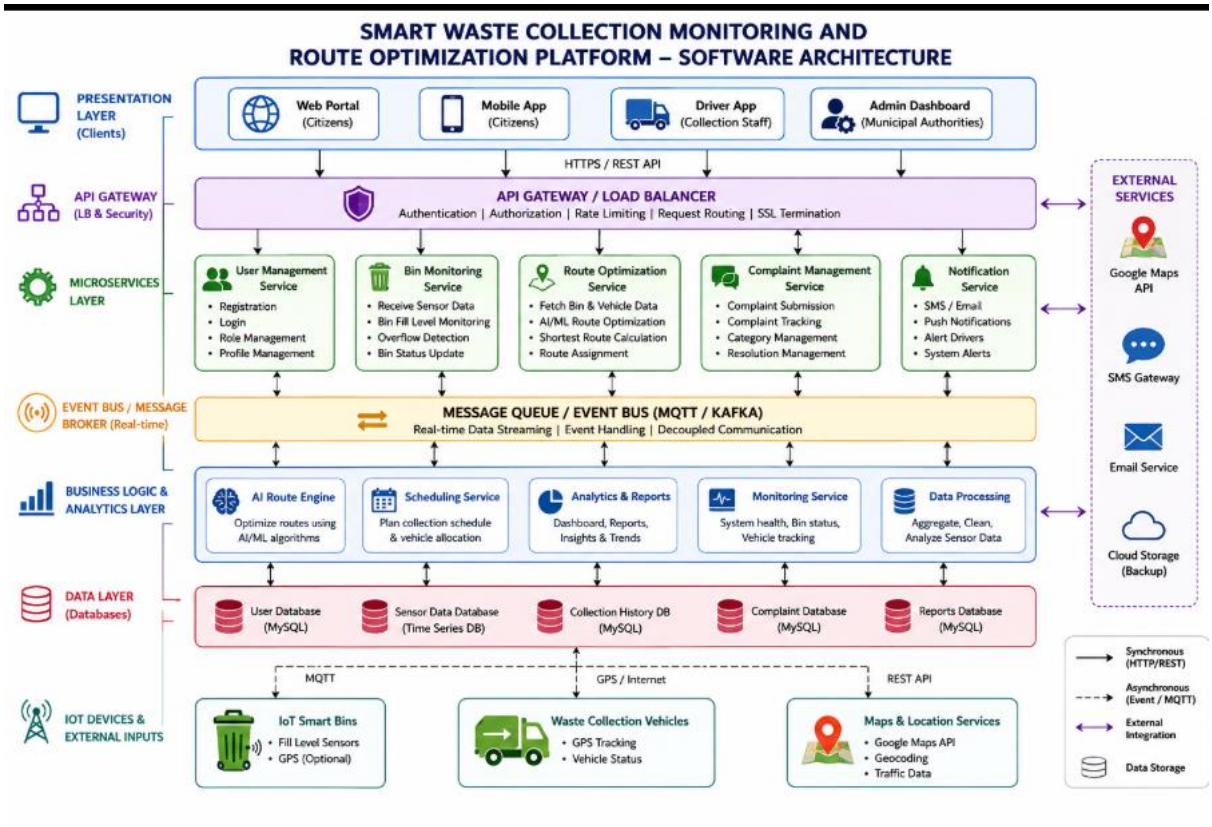
Reasons for Selecting Hybrid Architecture

- Supports real-time monitoring of smart bins.

- Efficiently handles IoT sensor events.
- Easily scales as the number of users and devices increases.
- Enables independent development and deployment of services.
- Improves fault tolerance by isolating service failures.
- Reduces maintenance complexity through modular design.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.



- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.

Presentation Layer

- Web Portal
- Mobile Application

- Driver Application
- Administrator Dashboard

API Gateway

- Receives all client requests
- Performs authentication and request routing
- Load balancing

Microservices Layer

- User Management Service
- Bin Monitoring Service
- Route Optimization Service
- Complaint Management Service
- Notification Service

Communication Mechanism

- REST APIs
- HTTPS
- MQTT/Event Bus (for IoT communication)

Business Logic Layer

- AI Route Optimization
- Waste Collection Scheduling
- Data Analytics
- Monitoring
- Report Generation

Database Layer

- User Database

- Sensor Data Database
- Collection History
- Complaint Database
- Reports Database

External Systems

- IoT Smart Bin Sensors
- GPS Vehicle Tracking
- Google Maps API
- Internet/Cloud Services

  - Use appropriate software architecture notations and labels.

- HTTPS – Secure communication between users and the system.
- REST APIs – Data exchange between applications and services.
- MQTT/Event Bus – Real-time communication from IoT smart bins.
- SQL Queries – Data storage and retrieval from databases.
- Google Maps API – Route calculation and location services.
- GPS Tracking – Live vehicle location updates for route optimization.

4. Explain Components and Their Interactions

  - Describe the purpose and responsibilities of each architectural component

1. Presentation Layer (Web Portal, Mobile App, Driver App, Admin Dashboard)

- Provides the user interface for citizens, drivers, and municipal administrators.
- Allows users to log in, monitor bin status, submit complaints, and view reports.
- Displays real-time waste collection information through dashboards.

2. API Gateway

- Serves as the main entry point for all client requests.
- Authenticates users and routes requests to the appropriate microservices.
- Provides load balancing, security, and request management.

3. User Management Service

- Manages user registration, login, and authentication.
- Maintains user profiles and role-based access control.
- Ensures secure access to system resources.

4. Bin Monitoring Service

- Receives real-time data from IoT smart bin sensors.
- Monitors bin fill levels and detects overflowing bins.
- Updates the system with the latest bin status.

5. Route Optimization Service

- Uses GPS and AI algorithms to generate the shortest collection routes.
- Optimizes vehicle movement to reduce travel time and fuel consumption.
- Updates routes based on live traffic and bin status.

6. Complaint Management Service

- Records complaints submitted by citizens.
- Tracks complaint status and assigns them to responsible authorities.
- Maintains complaint history for future reference.

7. Notification Service

- Sends SMS, email, and mobile push notifications.
- Alerts drivers about collection schedules and full bins.
- Notifies administrators about system events and emergencies.

8. Event Bus / Message Queue

- Transfers real-time events between services.
- Ensures fast and reliable communication.
- Supports asynchronous data processing.

9. Business Logic & Analytics Layer

- Processes sensor data and system operations.
- Generates reports, dashboards, and waste collection analytics.
- Supports AI-based decision-making and scheduling.

10. Database Layer

- Stores user details, sensor readings, complaints, reports, and collection history.
- Provides secure and reliable data storage.
- Supports data backup and recovery.

11. IoT Devices & External Systems

- Smart bin sensors monitor waste levels.
- GPS devices track collection vehicles.
- Google Maps API provides route and traffic information.
- SMS and Email services send notifications to users.
    - Explain how the components communicate and exchange data.
    - Smart bin IoT sensors collect fill-level data and send it to the Bin Monitoring Service using MQTT or the Internet.
    - The API Gateway receives requests from the web portal and mobile applications through REST APIs.
    - The Bin Monitoring Service updates the database and publishes events to the Message Queue when bins become full.
    - The Route Optimization Service retrieves bin locations from the database and GPS information from Google Maps API to calculate efficient routes.
    - The Notification Service sends alerts to drivers, administrators, and citizens through SMS, email, or push notifications.

- The Business Logic Layer processes data received from all services and generates reports for administrators.
- All services read from and write to the Database Layer using secure SQL queries.
- External services such as Google Maps API, GPS, and Cloud Storage exchange data through secure APIs.

- Illustrate the overall workflow of the system.

- Step 1

Smart waste bins continuously monitor the garbage fill level using IoT sensors.

- Step 2

Sensor data is transmitted to the cloud through MQTT or Internet communication.

- Step 3

The Bin Monitoring Service receives the sensor data and updates the database.

- Step 4

If a bin reaches its maximum capacity, the system automatically generates an event and sends it to the Message Queue.

- Step 5

The Route Optimization Service analyzes the locations of full bins, GPS data, and traffic conditions to calculate the shortest collection route.

- Step 6

The Notification Service sends alerts to the assigned waste collection driver and municipal administrator.

- Step 7

The driver follows the optimized route and collects waste from the identified bins.

- Step 8

After collection, the driver updates the collection status through the mobile application.

- Step 9

The system stores all collection details, complaints, and sensor data in the database.

- Step 10

Administrators view live dashboards, analytics, and reports to monitor system performance and improve future waste collection planning.

5. Discuss Quality Attributes

- Evaluate how the proposed architecture addresses the following aspects:

- Scalability

he proposed Hybrid Architecture is highly scalable because it supports the independent scaling of different services based on demand. As the number of smart bins, vehicles, or users increases, additional service instances can be deployed without affecting the entire system. Cloud infrastructure also enables the platform to expand across multiple cities while maintaining consistent performance.

Key Points

- Supports thousands of smart bins and users.
- Allows independent scaling of microservices.
- Easily expands to multiple cities.
- Handles increased data and network traffic efficiently

- Security

The architecture ensures strong security by implementing authentication, authorization, data encryption, and secure communication protocols. Only authorized users can access system resources, while sensitive information is protected during storage and transmission. API security and regular monitoring further reduce cybersecurity risks.

Key Points

- Secure user authentication and login.
- Role-based access control.
- HTTPS for secure communication.
- Data encryption during storage and transmission.
- Secure APIs protect against unauthorized access.

- Performance

The system is designed for high performance by processing IoT sensor data in real time and generating optimized routes quickly. Event-driven communication minimizes delays, while efficient database operations and cloud resources improve overall response time.

Key Points

- Real-time processing of sensor data.
- Fast route optimization using AI algorithms.
- Low response time for users.
- Efficient database access.

  - Reliability

The architecture improves reliability by ensuring continuous monitoring and fault tolerance. Independent microservices isolate failures, preventing one component from affecting the entire system. Backup mechanisms and automatic recovery further enhance system stability.

Key Points

- Continuous real-time monitoring.
- Fault isolation through microservices.
- Automatic failure recovery.
- Regular data backups.
- Accurate and consistent system operation.

  - Maintainability

The modular structure of the Hybrid Architecture makes the system easy to maintain and upgrade. Individual services can be modified, tested, or replaced without interrupting other components, reducing maintenance effort and improving software quality.

Key Points

- Modular and reusable components.
- Easy software updates and enhancements.

- Simplified debugging and testing.
- Independent deployment of services.
- Supports future feature additions
    - Availability

The platform provides high availability through cloud deployment, load balancing, and redundant servers. Automatic failover mechanisms ensure that services remain operational even if one server fails, allowing users to access the platform at any time.

Key Points

- 24×7 system availability.
- Cloud-based deployment.
- Load balancing improves uptime.
- Automatic failover during failures.
- Minimal service interruption and downtime.

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.

The Hybrid Architecture (Layered + Microservices + Event-Driven) was selected because it best meets the requirements of the Smart Waste Collection Monitoring and Route Optimization Platform. The system must handle real-time IoT sensor data, GPS-enabled vehicles, cloud services, notifications, and multiple users simultaneously. A hybrid approach combines the strengths of different architectural styles to provide better performance, flexibility, and scalability.

The Layered Architecture separates the presentation, business logic, and data layers, making the system organized and easy to maintain. Microservices Architecture divides the platform into independent services such as user management, bin monitoring, route optimization, and notifications, allowing each service to be developed and deployed independently. Event-Driven Architecture enables immediate processing of sensor events, ensuring real-time monitoring and faster response to overflowing bins.

Reasons for Choosing Hybrid Architecture

- Supports real-time IoT data processing.
- Separates the system into independent modules.
- Improves scalability and flexibility.
- Enables faster software development.
- Simplifies maintenance and testing.
- Provides better fault isolation.
- Supports cloud deployment.
- Integrates easily with external APIs.
- Delivers high performance for smart city applications.
- Suitable for future expansion.
    - Discuss the trade-offs considered while selecting the architecture.

While the Hybrid Architecture offers many benefits, certain trade-offs were considered before selecting it. Compared to a simple layered architecture, it is more complex to design and manage. However, the advantages of scalability, reliability, and flexibility outweigh the additional complexity.

Advantages

- Highly scalable for growing cities.
- Independent deployment of services.
- Better fault tolerance.
- Faster processing of real-time events.
- Easy integration with IoT devices and cloud platforms.
- Supports continuous updates without affecting the whole system.
- Improves overall system performance.
- Easier to add new features.

Disadvantages

- More complex architecture.
- Higher initial development cost.
- Requires service orchestration and monitoring.
- Increased deployment complexity.
- More challenging debugging across multiple services
    - Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

The selected architecture is designed to support future growth and technological advancements. Since each microservice works independently, new features can be added without modifying the existing system. The platform can easily integrate with additional Smart City applications, cloud platforms, AI services, and third-party APIs.

In the future, advanced technologies such as AI-based waste prediction, smart recycling management, electric vehicle tracking, carbon emission monitoring, and predictive maintenance can be added as separate services. The modular architecture also simplifies software updates, testing, and maintenance, ensuring the platform remains efficient and reliable over the long term.

Future Benefits

- Easy integration with Smart City systems.
- Supports AI and Machine Learning features.
- Allows addition of new microservices.
- Simplifies software upgrades.
- Reduces maintenance effort.
- Supports cloud expansion.
- Easy integration with new IoT devices.
- Ensures long-term scalability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.

Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25